

目录

第一章 绪论..... 4

1.1 研究背景与意义..... 4

1.1.1 网络安全的重要性 4

1.1.2 协议分析在教学与运维中的作用 4

1.1.3 现有工具的局限性与自主开发的必要性 4

1.2 国内外研究现状..... 5

1.2.1 网络嗅探技术的发展 5

1.2.2 基于 Python 的网络工具生态..... 5

1.3 本文主要工作 5

1.4 全文结构安排 6

第二章 相关技术综述 6

2.1 Python 语言及其优势 6

2.2 Scapy 网络协议库 6

2.2.1 工作原理..... 7

2.2.2 协议分层模型..... 7

2.3 PySide6 图形界面框架..... 7

2.3.1 信号与槽机制 (Signals and Slots)..... 7

2.3.2 QThread 多线程..... 8

2.4 Matplotlib 数据可视化库..... 8

2.4.1 在 GUI 中嵌入图表..... 8

2.4.2 图表类型..... 8

第三章 系统需求分析 9

3.1 功能性需求..... 9

3.1.1 网卡识别与选择 10

3.1.2 数据包实时捕获与控制 10

3.1.3 多协议深度解析	10
3.1.4 数据可视化与交互	11
3.1.5 统计分析报表	11
3.2 非功能性需求	11
3.3 系统用例分析	11
第四章 系统总体设计	12
4.1 系统架构设计	12
4.2 模块划分与功能设计	13
4.2.1 抓包引擎模块 (core/sniffer.py)	13
4.2.2 协议解析模块 (utils/parser.py)	13
4.2.3 主界面交互模块 (ui/main_window.py)	14
4.2.4 数据分析模块 (ui/analysis_window.py)	14
4.3 核心业务流程设计	14
4.3.1 数据包捕获与处理流程	14
第五章 详细设计与实现	15
5.1 抓包引擎的实现 (Sniffer Engine)	15
5.1.1 多线程并发模型	15
5.1.2 Scapy 捕获逻辑	16
5.2 协议解析器的实现 (Protocol Parser)	16
5.2.1 逐层解析架构	17
5.2.2 字段汉化与格式化	17
5.2.3 地理位置增强 (Geo-IP)	18
5.2.4 载荷可视化 (Hex Dump)	18
5.3 主界面的实现 (Main Interface)	18
5.3.1 列表渲染与色彩规则 (Coloring Rules)	18
5.3.2 界面联动逻辑	19
5.4 统计分析功能的实现 (Statistical Analysis)	19

5.4.1 数据聚合算法.....	19
第六章 系统测试与运行展示.....	20
6.1 测试环境配置	20
6.2 功能测试	20
6.2.1 系统启动与网卡选择.....	20
6.2.2 实时数据包捕获.....	21
6.2.3 协议深度解析与汉化展示.....	21
6.2.4 流量统计分析.....	22
6.3 性能测试	23
第七章 总结与展望	23
7.1 项目总结	23
7.2 存在的不足.....	24
7.3 改进方向	24

基于 Python 的网络流量捕获与可视化分析系统设计与实现

第一章 绪论

1.1 研究背景与意义

随着互联网技术的飞速发展，计算机网络已成为现代社会基础设施的重要组成部分。网络通信协议的复杂性以及网络安全威胁的多样化，使得对网络流量进行实时监控、捕获与分析变得尤为重要。无论是在网络故障排查、性能优化，还是在网络安全防御（如入侵检测、敏感数据泄露审计）中，网络嗅探（Network Sniffing）技术都扮演着核心角色。

1.1.1 网络安全的重要性

在开放的网络环境中，数据通常以数据包（Packet）的形式传输。如果缺乏有效的加密手段，诸如 HTTP、FTP、Telnet 等明文协议传输的敏感信息（如账号、密码、个人隐私）极易被中间人截获。通过抓包工具，安全研究人员可以复现攻击路径，分析恶意软件的通信特征，从而制定有效的防御策略。

1.1.2 协议分析在教学与运维中的作用

对于计算机相关专业的学生及网络工程师而言，TCP/IP 协议栈通常是抽象且复杂的。通过开发并使用可视化的抓包工具，能够将抽象的“三次握手”、“四次挥手”、DNS 解析过程以直观的图形化界面展示出来，极大地降低了理解门槛。同时，在网络运维中，抓包工具是定位网络延迟、丢包、连接超时等疑难杂症的“听诊器”。

1.1.3 现有工具的局限性与自主开发的必要性

目前业界已存在如 Wireshark、Tcpdump 等成熟且强大的商业级或开源抓包工具。然而，这些工具通常功能过于庞大，操作界面复杂，且通过 C/C++ 编写，二次开发难度较高。对于特定场景下的轻量级分析需求，或者为了深入理解网络底层机制，利用现代脚本语言（如 Python）设计并实现一个定制化的网络抓包与分析系统具有重要的实践意义。

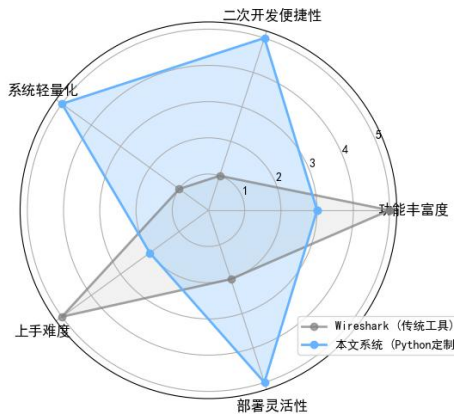


图 1-1 现有工具与本系统特性对比分析

1.2 国内外研究现状

1.2.1 网络嗅探技术的发展

早期的网络嗅探技术主要依赖于 Unix 系统下的 BPF (Berkeley Packet Filter) 机制，通过将网卡设置为“混杂模式” (Promiscuous Mode) 来接收所有流经网卡的数据帧。随着技术的发展，Libpcap (Linux/Unix) 和 WinPcap/Npcap (Windows) 成为了行业标准的底层驱动库，它们提供了独立于操作系统的直接访问网络数据链路层的能力。

1.2.2 基于 Python 的网络工具生态

近年来，Python 因其语法简洁和丰富的第三方库生态，逐渐成为网络安全和自动化运维的首选语言。

Scapy: 作为一个强大的交互式数据包处理程序，Scapy 不仅能抓包，还能构造、发送、解码数据包，被广泛应用于网络扫描和攻击模拟。

dpkt / pypcap: 提供了更底层的解析能力，追求高性能解析。

Impacket: 专注于网络协议 (如 SMB, MSSQL, NetBIOS) 的底层访问。

相比于传统的 C/C++ 开发模式，基于 Python 的开发大幅缩短了开发周期，并且配合 PySide/PyQt 等 GUI 框架，能够快速构建出跨平台的图形化应用。

1.3 本文主要工作

本文设计并实现了一个基于 Python 的可视化网络流量抓包与分析系统。该系统模仿了 Wireshark 的核心交互逻辑，旨在提供一个轻量、易用且功能完备的流量分析平台。本文的主要工作包括：

系统架构设计：采用 MVC (模型-视图-控制器) 设计模式，将底层抓包逻辑、协议解析逻辑与前端界面展示分离，提高了代码的可维护性和扩展性。

多协议深度解析：实现了对以太网 (Ethernet)、IP、ARP、ICMP、TCP、UDP 等基础协议的头部字段解析，并进一步支持 DNS、HTTP、TLS (握手阶段) 等应用层协议的识别与展示。

用户界面开发：利用 PySide6 框架构建了响应式的图形用户界面 (GUI)，实现了数据包列表的实时滚动刷新、协议类型的色彩区分 (Coloring Rules) 以及详细信息的树状视图展示。

数据可视化分析：集成了 Matplotlib 绘图库，实现了对捕获流量的统计分析功能，包括协议占比饼图、活跃源 IP 排名柱状图。

地理位置增强：集成了 IP 地理位置查询接口，能够实时显示通信对端的国家、城市及运营商信息。

1.4 全文结构安排

第一章：介绍研究背景、意义及主要工作。

第二章：综述项目所涉及的关键技术（Python, Scapy, PySide6, Matplotlib）。

第三章：进行详细的系统需求分析。

第四章：阐述系统的总体架构设计与模块划分。

第五章：详细介绍各核心模块的具体实现代码与逻辑。

第六章：展示系统运行效果及功能测试结果。

第七章：总结全文并展望未来工作。

第二章 相关技术综述

2.1 Python 语言及其优势

Python 是一种解释型、面向对象、动态数据类型的高级程序设计语言。本项目选择 Python 作为开发语言，主要基于以下考量：

开发效率高：Python 简洁的语法使得开发者可以将精力集中在业务逻辑（协议解析）而非底层内存管理上。

丰富的网络库：Python 拥有 Scapy、Socket 等成熟的网络编程库，能够轻松处理二进制数据流。

强大的 GUI 支持：通过 PySide6，Python 可以调用工业级的 Qt 框架，构建美观且功能强大的桌面应用。

2.2 Scapy 网络协议库

Scapy 是本项目核心的“抓包引擎”。它是一个强大的 Python 模块，能够伪造、解码、发送和捕获网络数据包。

2.2.1 工作原理

Scapy 在底层利用操作系统的原始套接字 (Raw Socket) 或通过 Libpcap/Npcap 驱动直接与网卡交互。当捕获到一个数据包时, Scapy 会将其视为一串二进制流, 并根据协议栈的层次结构 (Layer) 进行逐层解码。

2.2.2 协议分层模型

在 Scapy 中, 一个复杂的数据包被表示为不同协议层的叠加。例如, 一个 TCP 数据包在 Scapy 中表示为: Ether() / IP() / TCP() / Payload 这种对象模型使得开发者可以通过 packet[TCP].sport 这样直观的方式直接读取任意协议层的字段, 极大地简化了代码编写复杂度。本项目利用 Scapy 的 sniff() 函数实现数据包的持续捕获, 并利用其内置的 summary() 和 show() 方法作为解析逻辑的基础。

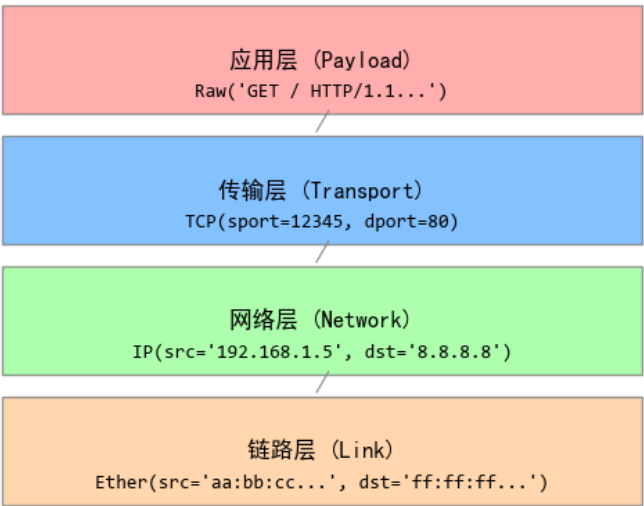


图 2-1 Scapy 协议分层对象模型示意图

图 2-1 Scapy 协议分层对象模型示意图

2.3 PySide6 图形界面框架

PySide6 是 Qt 公司官方提供的 Qt 6 框架的 Python 绑定。Qt 是目前全球范围内应用最广泛的 C++ 跨平台 GUI 库之一。

2.3.1 信号与槽机制 (Signals and Slots)

这是 Qt 框架的核心特性, 用于对象间的通信。在本项目中, 底层抓包线程 (Sniffer Thread) 与主界面 (Main Window) 之间完全解耦。当后台线程捕获到一个数据包时, 通过发射一个 Signal (信号), 主界面预先连接的 Slot (槽函数) 会自动被

调用，从而更新 UI。这种机制保证了界面的响应性，避免了因后台处理耗时而导致的界面卡死。

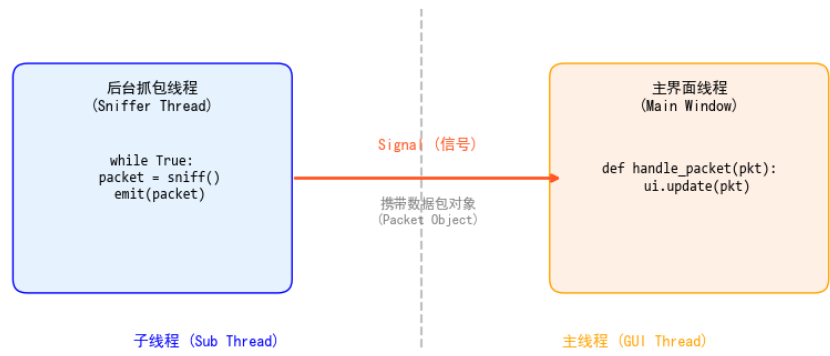


图 2-2 基于 Qt 信号槽的多线程通信机制

图 2-2 基于 Qt 信号槽的多线程通信机制

2.3.2 QThread 多线程

为了防止网络抓包的阻塞操作（I/O Blocking）影响用户界面的渲染，本项目使用了 QThread 类。抓包任务被封装在一个独立的子线程中运行，充分利用了多核 CPU 的性能，实现了数据采集与界面显示的并发执行。

2.4 Matplotlib 数据可视化库

Matplotlib 是 Python 生态中最著名的数据可视化库，常用于科学计算和数据分析领域。

2.4.1 在 GUI 中嵌入图表

虽然 Matplotlib 通常用于生成静态图片，但它提供了 FigureCanvasQTAgg 后端接口，允许将生成的图表作为普通的 Qt 控件（Widget）嵌入到 PySide6 的窗口中。

2.4.2 图表类型

本项目利用 Matplotlib 绘制了两种核心图表：

饼图 (Pie Chart): 展示 TCP、UDP、ARP 等协议在总流量中的百分比。

柱状图 (Bar Chart): 展示发送数据包数量最多的 Top 5 IP 地址。

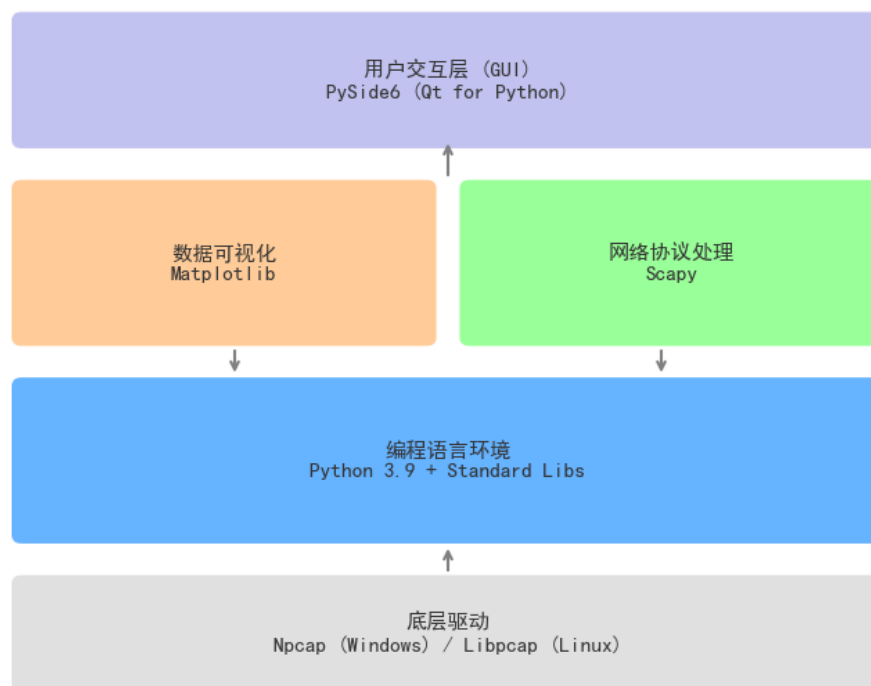


图 2-3 系统技术架构层次图

图 2-3 系统技术栈架构图

第三章 系统需求分析

3.1 功能性需求

本系统的主要目标是构建一个仿 Wireshark 的轻量级网络流量分析工具，具体功能需求图 3-1 所示：

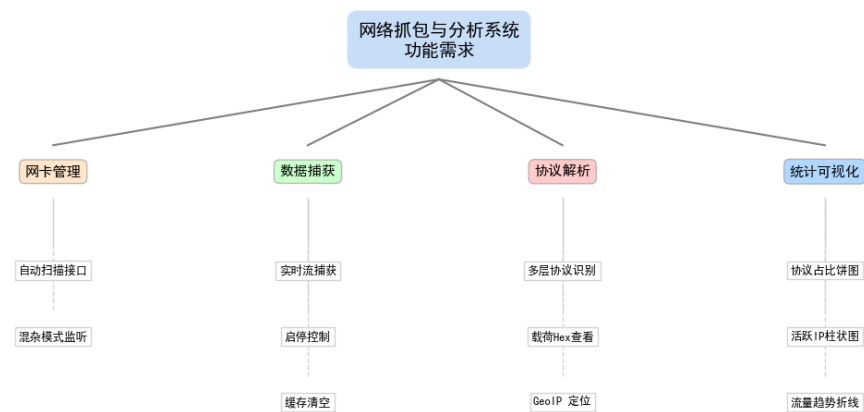


图 3-1 系统功能需求层次分解图

图 3-1 系统功能需求层次分解图

3.1.1 网卡识别与选择

系统启动后，应能自动扫描当前主机上所有可用的网络接口（包括以太网适配器、无线网卡、虚拟网卡等）。用户可以通过下拉菜单选择目标网卡，系统需支持对选定网卡的混杂模式（Promiscuous Mode）监听。

3.1.2 数据包实时捕获与控制

开始/停止控制：用户可随时开启或暂停抓包任务。

清空缓存：支持一键清空当前捕获的数据列表，释放内存。

实时显示：当捕获到新的数据包时，界面列表应自动滚动并刷新，显示数据包的摘要信息（编号、时间戳、源/目的 IP、协议类型、长度、简要信息）。

3.1.3 多协议深度解析

系统不仅需要识别基础的网络层和传输层协议，还需对应用层协议进行解析。具体支持的协议包括：

链路层：Ethernet II（源/目的 MAC 地址、类型）。

网络层：IPv4（版本、头部长度、TTL、校验和）、ARP（请求/应答、MAC-IP 映射）。

传输层：TCP（端口、序列号、标志位、窗口大小）、UDP（端口、长度）。

应用层：

DNS：识别查询与响应、解析域名（Domain Name）、事务 ID。

HTTP：识别请求方法（GET/POST）、Host 主机名、User-Agent。

TLS/SSL：识别握手包（Client Hello/Server Hello）及加密应用数据。

3.1.4 数据可视化与交互

色彩区分：根据协议类型（TCP/UDP/ICMP/ARP 等）自动为列表行添加不同的背景色，以使用户快速区分流量类型。

详细信息展示：用户点击列表中的数据包包时，底部应展开树状视图，分层显示该数据包各协议头的详细字段值。

载荷查看：支持以“十六进制 + ASCII 字符”对照的方式查看数据包的 Payload（有效载荷）。

地理位置增强：支持通过外部 API 查询公网 IP 的地理归属地（国家/城市/ISP）。

3.1.5 统计分析报表

系统应提供独立的统计分析窗口，对捕获的流量进行宏观分析：

协议分布统计：以饼图形式展示各协议数据包的数量占比。

活跃主机统计：以柱状图形式展示发送数据包最多的 Top 5 源 IP 地址。

3.2 非功能性需求

性能需求：在百兆或千兆网络环境下，系统应能稳定捕获数据包，且界面不应出现明显的卡顿或无响应（ANR）现象。这要求抓包逻辑必须与界面渲染逻辑分离（多线程）。

易用性需求：界面设计应简洁直观，操作逻辑应符合用户习惯（参考 Wireshark 布局）。

兼容性需求：系统需在 Windows 10/11 操作系统上稳定运行，并兼容 Npcap 驱动。

3.3 系统用例分析

本系统的主要使用者为网络管理员或学生，主要用例包括：

配置抓包环境：选择网卡。

执行抓包任务：开始、停止、清空。

查看数据包详情：点击列表，查看协议树和载荷。

进行统计分析：打开统计窗口，查看图表。

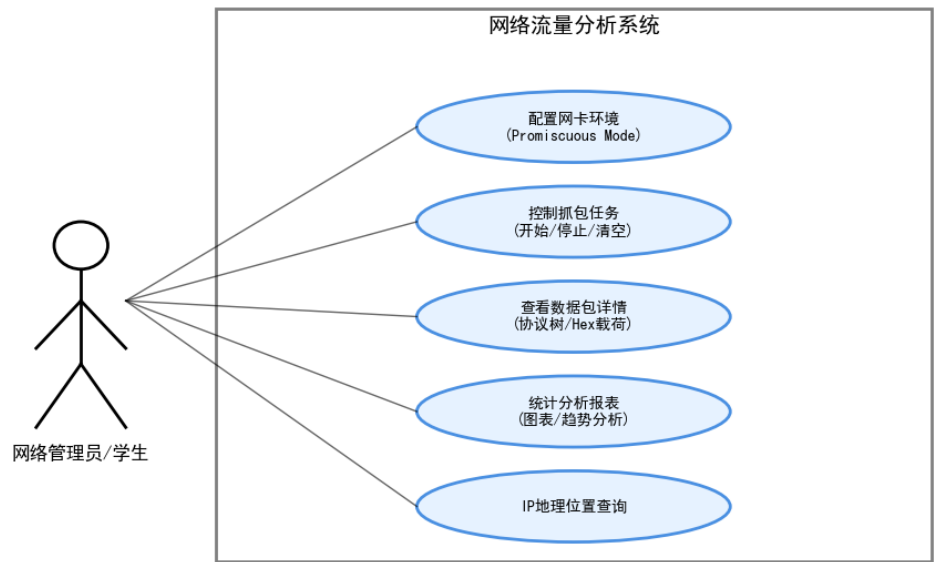


图 3-2 系统 UML 用例图

图 3-2 系统 UML 用例图

第四章 系统总体设计

4.1 系统架构设计

为了降低各功能模块之间的耦合度，提高代码的可读性和可维护性，本系统采用经典的 MVC (Model-View-Controller) 变体架构，并结合 生产者-消费者 模型进行设计。

系统自顶向下分为三层：

表现层 (View)：由 ui 模块构成，负责界面的绘制、用户交互响应（点击、滚动）以及图表的渲染。

逻辑控制层 (Controller/Logic)：由 main_window.py 和 analysis_window.py 中的逻辑代码构成，负责协调底层数据与前端展示，处理信号槽连接。

数据处理层 (Model)：由 core 和 utils 模块构成，负责底层的数据包捕获 (Sniffer)、协议解析 (Parser) 以及数据存储。

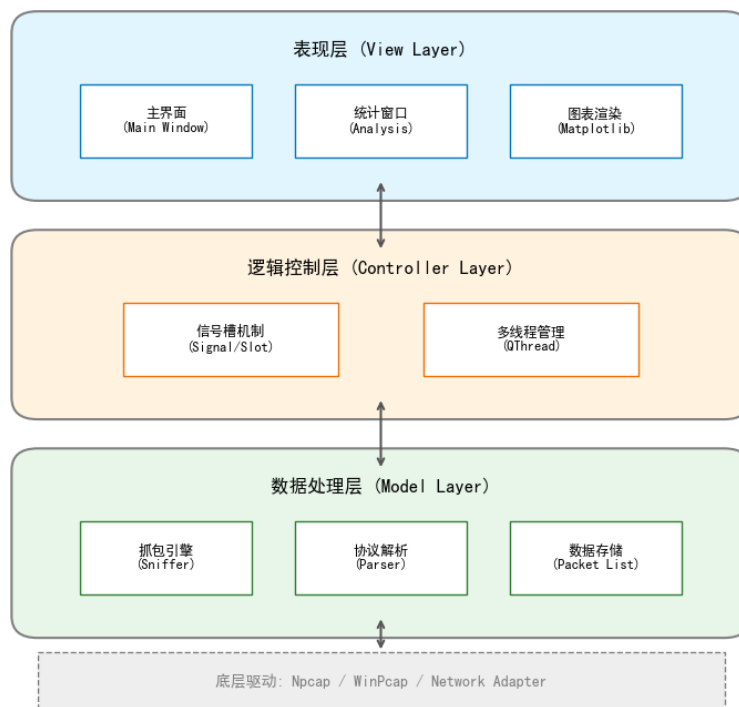


图 4-1 系统分层架构设计图

图 4-1 系统分层架构设计图

4.2 模块划分与功能设计

系统代码结构清晰，主要划分为以下四个核心模块：

4.2.1 抓包引擎模块 (core/sniffer.py)

这是系统的“发动机”。该模块封装了一个继承自 QThread 的 SnifferThread 类。

功能：在后台子线程中调用 Scapy 的 sniff() 函数进行阻塞式抓包。

通信机制：定义了 packet_captured 信号。每当捕获到一个数据包，立即通过 emit() 方法将原始数据包对象和预处理的摘要信息发送给主线程，实现了数据采集与界面显示的解耦。

4.2.2 协议解析模块 (utils/parser.py)

这是系统的“翻译官”。该模块包含静态工具类 PacketParser。

摘要提取：get_summary_info() 方法负责快速提取列表显示所需的源/目的 IP、协

议、长度等信息。

深度解析：get_tree_details() 方法负责将 Scapy 对象逐层拆解 (Ethernet -> IP -> TCP/UDP -> App)，并将字段名转换为中文，构建用于 QTreeWidget 显示的层级数据结构。

辅助功能：包含 IP 地理位置查询 (get_ip_location) 和 Payload 格式化 (format_payload) 功能。

4.2.3 主界面交互模块 (ui/main_window.py)

这是系统的“控制台”。

列表渲染：使用 QTableWidgetItem 展示数据包列表，并根据协议类型实现了 Coloring Rules (如 TCP 显示紫色，UDP 显示杏色)。

交互逻辑：实现了网卡列表的加载、抓包线程的启停控制、以及点击列表行触发详情展示的逻辑。

4.2.4 数据分析模块 (ui/analysis_window.py)

这是系统的“仪表盘”。

数据聚合：使用 Python 的 Counter 类对捕获到的数据包列表进行快速统计。

图表绘制：利用 Matplotlib 的 FigureCanvasQTAgg 后端，在 Qt 窗口中动态绘制饼图、柱状图。

4.3 核心业务流程设计

4.3.1 数据包捕获与处理流程

系统的核心运行流程是一个典型的异步事件驱动过程：

用户点击“开始抓包”，主线程实例化并启动 SnifferThread。

子线程调用 Scapy 接口监听网卡。

生产者：子线程捕获数据包，调用 PacketParser 进行初步解析，并通过 Signal 发射数据。

消费者：主线程的 Slot 函数接收信号，将数据追加到内存列表，并更新 UI 表格。

用户点击“停止抓包”，子线程终止循环并释放资源。

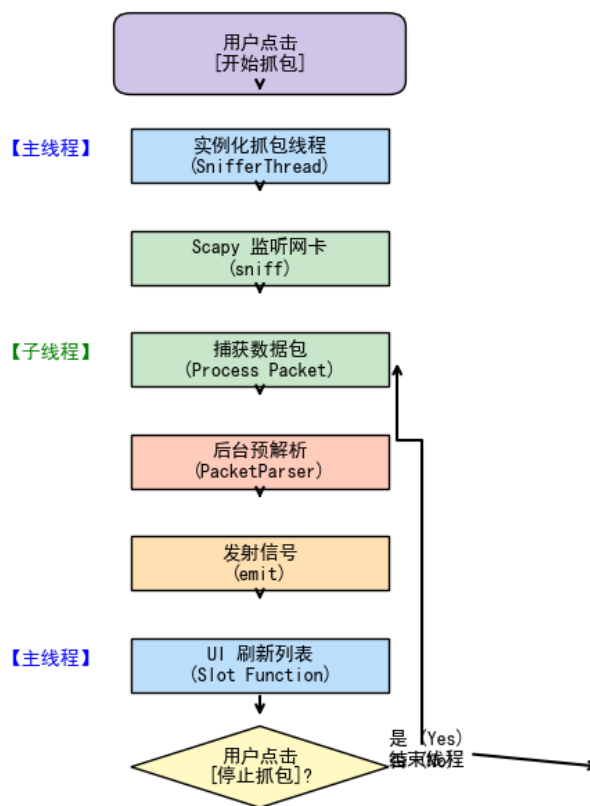


图 4-2 数据捕获与处理核心流程图

图 4-2 数据捕获与处理核心流程图

第五章 详细设计与实现

5.1 抓包引擎的实现 (Sniffer Engine)

抓包引擎是系统的底层核心，位于 `core/sniffer.py` 模块。为了解决网络抓包过程中 I/O 阻塞导致界面卡死 (Freezing) 的问题，本系统利用 PySide6 的 QThread 多线程技术实现了数据采集与界面展示的异步分离。

5.1.1 多线程并发模型

SnifferThread 类继承自 QThread，重写了 `run()` 方法作为子线程的入口。在主线程初始化界面后，启动该子线程进行后台监听。

信号定义：为了安全地跨线程传递数据，定义了自定义信号 `packet_captured`。

Python

信号携带两个参数：原始数据包对象、预处理的摘要信息

```
packet_captured = Signal(object, object)
```

注：这里采用了一种性能优化策略。传统的做法是在 UI 线程解析数据包，但这会导致界面在高流量下卡顿。本系统改进了逻辑，将耗时的“摘要解析”工作前移至子线程完成，主线程仅负责渲染，极大提升了流畅度。

5.1.2 Scapy 捕获逻辑

在 `run()` 方法中，调用 Scapy 的核心函数 `sniff()`。

`iface`：指定用户在下拉框中选择的网卡接口。

`prn`：回调函数 `process_packet`，每捕获一个包就调用一次。

`store=0`：关键参数，指示 Scapy 不要在内存中缓存所有抓到的包，而是处理完即丢弃引用，防止长时间运行导致内存溢出（OOM）。

Python

```
def process_packet(self, packet):
```

```
    if self.running:
```

```
        try:
```

```
            # 1. 在后台线程预先解析摘要（减轻主线程负担）
```

```
            summary_info = PacketParser.get_summary_info(packet, 0)
```

```
            # 2. 发射信号，将数据“推”给主界面
```

```
            self.packet_captured.emit(packet, summary_info)
```

```
        except Exception:
```

```
            pass
```

5.2 协议解析器的实现 (Protocol Parser)

`utils/parser.py` 是系统的“业务逻辑层”，负责将晦涩的二进制数据翻译为用户可读的中文信息。

5.2.1 逐层解析架构

PacketParser 类采用静态方法 `get_tree_details()`，利用 Scapy 的 `haslayer()` 方法判断协议类型，自底向上构建层级数据：

- 链路层：提取 Ether 层，解析源 MAC 和目的 MAC 地址。
- 网络层：提取 IP 层，解析版本号、TTL、校验和等。
- 传输层：根据 proto 字段分支处理 TCP 或 UDP。
- 应用层：进一步识别 DNS、HTTP 或 TLS 载荷。

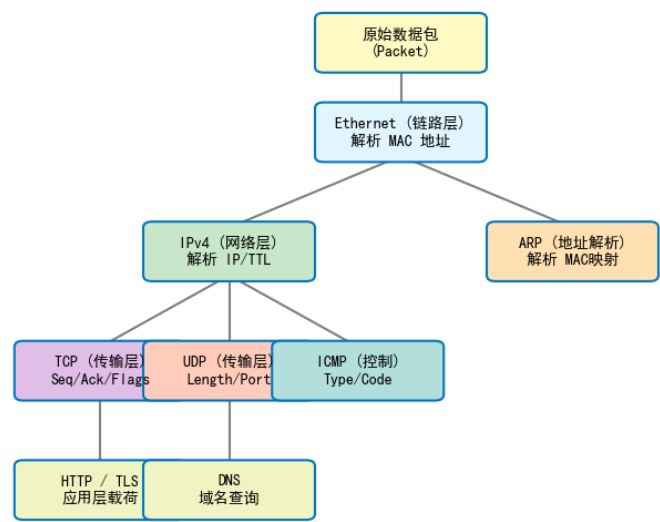


图 5-1 协议逐层解析逻辑树 (Protocol Parser Tree)

图 5-1 协议逐层解析逻辑树

5.2.2 字段汉化与格式化

为了提升用户体验，系统对所有技术术语进行了汉化处理。

TCP 标志位解析：利用 `packet.printf('%TCP.flags%')` 获取标志位的字符表示（如 SA、PA），并结合十六进制值显示。

DNS 标志位计算：由于 Scapy 的 DNS 层将标志位拆解为多个字段，本系统通过位运算手动重建了 16 位的 Flags 值，以便以 0x8180 等标准形式展示。

Python

```
# 手动合成 DNS Flags
```

```
flags_val = (dns.qr << 15) | (dns.opcode << 11) | ... | dns.rcode
```

5.2.3 地理位置增强 (Geo-IP)

在 `get_ip_location()` 方法中，系统通过 `requests` 库调用 `http://ip-api.com` 接口。

逻辑优化：自动过滤 192.168.x.x 等内网 IP，避免无效查询。

超时保护：设置 `timeout=3.0`，防止因网络波动导致的查询阻塞。

5.2.4 载荷可视化 (Hex Dump)

`format_payload()` 方法模拟了 Wireshark 的底层视图。它将 Raw 层的二进制数据同时转换为 十六进制字符串（左侧）和 ASCII 可打印字符（右侧，不可见字符替换为 `.`），方便安全人员分析明文通信内容。

【建议插入图片：代码片段截图 (Parser 类的汉化部分)】

5.3 主界面的实现 (Main Interface)

`ui/main_window.py` 负责界面的构建与交互逻辑。

5.3.1 列表渲染与色彩规则 (Coloring Rules)

为了实现“一眼区分协议”的效果，系统在 `handle_packet` 槽函数中实现了严格的色彩映射逻辑，完美复刻了 Wireshark 的经典配色：

TCP (紫色): `QColor(216, 191, 216)` —— 表示可靠传输。

UDP (杏色): `QColor(255, 218, 185)` —— 表示不可靠传输或流媒体。

HTTP/DNS (黄色): `QColor(255, 255, 224)` —— 表示应用层查询。

ICMP (绿色): `QColor(144, 238, 144)` —— 表示网络控制消息（如 Ping）。

Error/ARP (深黄): `QColor(238, 238, 0)` —— 表示广播或警示。

系统预设色彩映射规则 (Coloring Rules)

TCP 协议	淡紫色 (Purple)
UDP 协议	杏仁色 (Apricot)
HTTP / DNS	浅黄色 (Light Yellow)
ICMP 协议	亮绿色 (Green)
ARP / Error	深黄色 (Dark Yellow)

图 5-2 列表行背景色映射表

图 5-2 列表行背景色映射表

5.3.2 界面联动逻辑

系统采用了 QTableWidgetItem (列表) 与 QTreeWidgetItem (树图) 的联动设计。

信号连接: self.table.cellClicked.connect(self.on_table_click)。

处理逻辑: 当用户点击列表中的某一行时, 程序根据行索引 (Row Index) 从内存列表 self.packets 中检索出对应的原始数据包对象, 即时调用解析器生成详情树, 并填充到底部区域。这种“懒加载” (Lazy Loading) 策略避免了预先解析所有详情带来的内存浪费。

5.4 统计分析功能的实现 (Statistical Analysis)

ui/analysis_window.py 模块集成了 Matplotlib, 通过 FigureCanvasQTAgg 将图表嵌入 Qt 窗口。

5.4.1 数据聚合算法

利用 Python 的 collections.Counter 高效统计数据:

协议统计: 遍历数据包列表, 通过 pkt.haslayer() 累加各协议数量。

Top IP 统计: 提取所有 IP 包的 src 字段, 计算出现频率最高的 5 个地址。

第六章 系统测试与运行展示

6.1 测试环境配置

操作系统：Windows 11 Professional (64-bit)

处理器：Intel Core i7 / AMD Ryzen 7

内存：16 GB RAM

开发环境：Python 3.9.13

核心依赖库：

PySide6 (6.x) - 界面框架

Scapy (2.5.0) - 网络核心

Matplotlib (3.x) - 图表绘制

Npcap (1.7x) - 底层驱动 (WinPcap 兼容模式)

6.2 功能测试

6.2.1 系统启动与网卡选择

系统启动后，主界面加载迅速。顶部控制栏的下拉菜单能够自动扫描并列出当前主机的所有网络接口，包括物理网卡（如 Realtek PCIe GbE, Wi-Fi 6 AX200）及虚拟网卡（如 VMWare, VPN 适配器）。

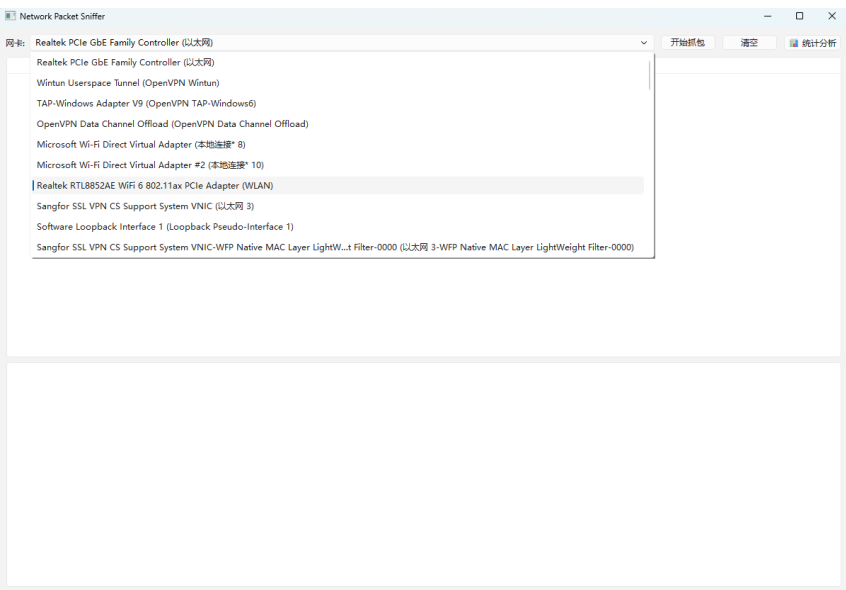


图 6-1 系统启动与网卡选择

6.2.2 实时数据包捕获

选择 Wi-Fi 网卡并点击“开始抓包”后，系统成功捕获了流经网卡的数据流。

列表刷新：中央列表区域实时滚动显示捕获的数据包。

色彩区分：系统根据预设的色彩规则对不同协议进行了高亮显示：

TCP (紫色)：显示了 Seq/Ack 序列号及标志位。

UDP (杏色)：显示了源端口与目的端口。

ICMP (绿色)：清晰识别了 Ping 请求与应答。

TLS/SSL (深紫)：识别出 Application Data 加密流量。

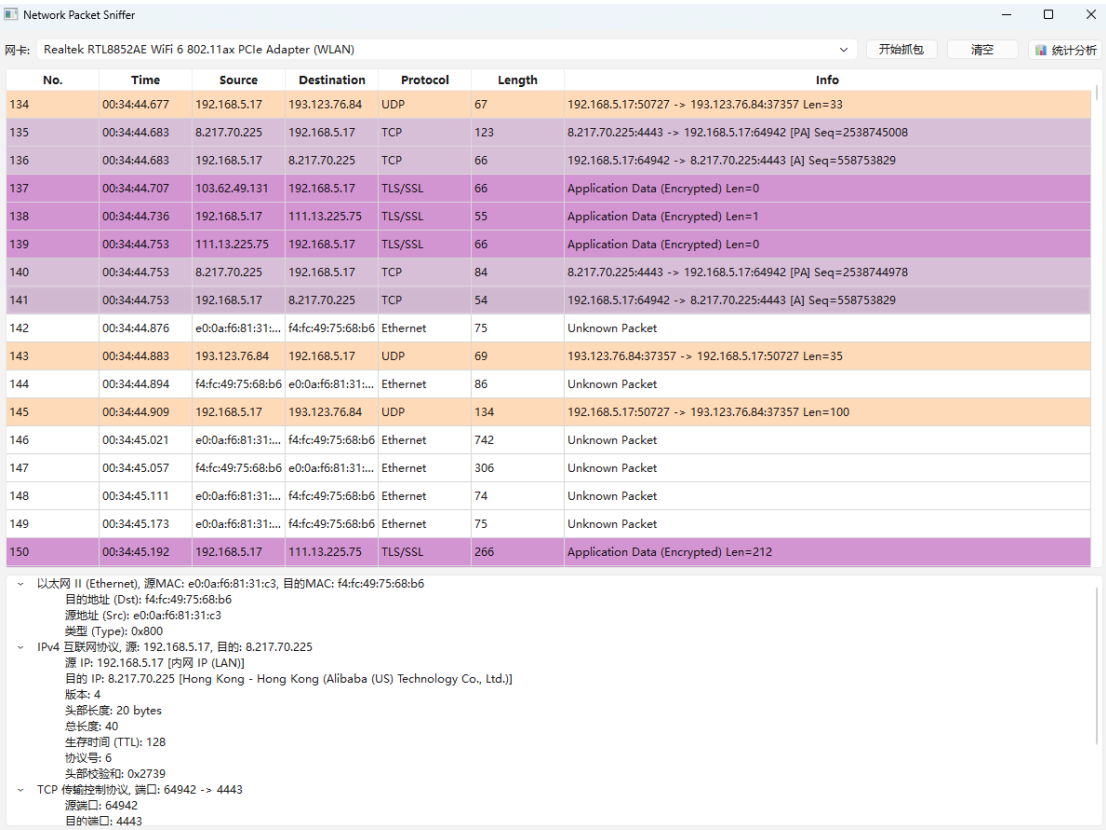


图 6-2 实时数据包捕获

6.2.3 协议深度解析与汉化展示

点击列表中的任意数据包，底部的树状视图（Tree View）能够正确展开，并显示汉化后的详细字段信息。

地理位置查询：测试显示，对于公网 IP，系统成功调用 API 显示了归属地（如 United States 或 中国 - Jinan）。

Payload 查看：底部的 Hex Dump 视图清晰展示了数据包的十六进制与 ASCII 内容，验证了对应用层载荷的解析能力。

35976	00:39:49:289	202.118.48.249	192.168.5.17	TLS/SSL	1494	Application Data (Encrypted) Len=1440
以太网 II (Ethernet), 源MAC: f4:fc:49:75:68:b7, 目的MAC: e0:0a:f6:81:31:c3 目的地址 (Dst): e0:0a:f6:81:31:c3 源地址 (Src): f4:fc:49:75:68:b7 类型 (Type): 0x800 IPv4 互联网协议, 源: 202.118.48.249, 目的: 192.168.5.17 源 IP: 202.118.48.249 [中国 - Haidian (CERNET)] 目的 IP: 192.168.5.17 [内网 IP (LAN)] 版本: 4 头部长度: 20 bytes 总长度: 1480 生存时间 (TTL): 51 协议号: 6 头部校验和: 0xe96b TCP 传输控制协议, 端口: 443 -> 61007 源端口: 443 目的端口: 61007 序列号 (Seq): 2432456432 确认号 (Ack): 324904536 标志位 (Flags): 0x10 (A) 窗口大小: 553 校验和: 0x40ea 数据载荷 (1440 字节) 十六进制: 85 55 c9 1e 95 24 9c b0 ee 83 79 e6 1a b4 e1 71 76 ad dd 39 5a 7c 19 58 42 ea 61 3a ef 30 46 32 73 d... ASCII文本: .U...\$...y...qv..9Z].XB.a:0Fz\$.z...N.nG).D.....						

图 6-3 协议深度解析

6.2.4 流量统计分析

在抓取一定量的数据后，点击“统计分析”按钮，弹出独立的分析窗口。

协议占比图：准确统计了 TCP/UDP/ARP 的分布比例。

活跃 IP 榜：成功识别出当前网络中发送数据最多的主机（包括本机 IP 和远程服务器 IP）。

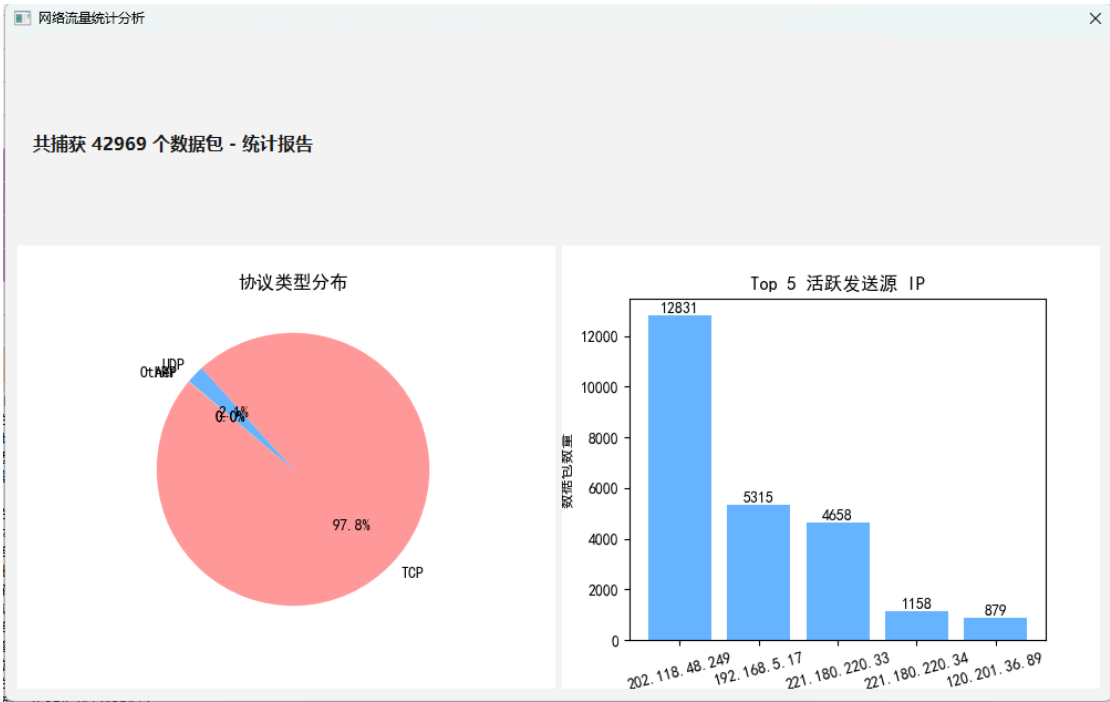


图 6-4 流量统计分析

6.3 性能测试

在开启视频流媒体（高带宽占用）的场景下进行压力测试：

稳定性：系统连续运行 30 分钟未出现崩溃或内存溢出（OOM）。

响应性：得益于多线程架构，即使每秒捕获数百个数据包，主界面依然保持响应，点击停止按钮可立即响应，未出现“程序无响应”的假死现象。

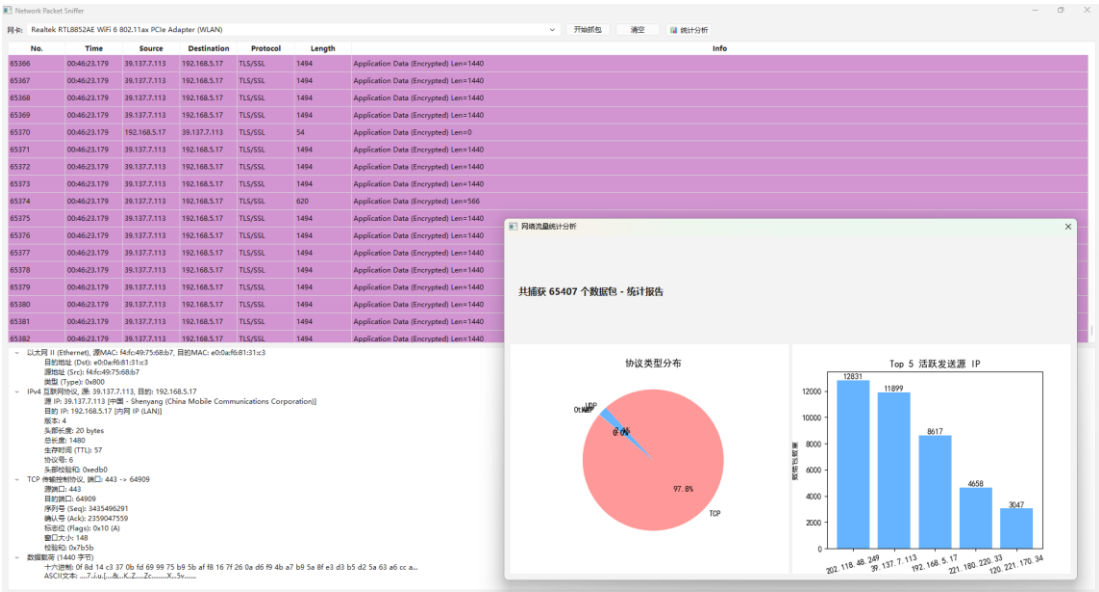


图 6-5 性能测试

第七章 总结与展望

7.1 项目总结

本文设计并实现了一款基于 Python 的网络流量抓包与分析系统。该项目综合运用了计算机网络协议原理、Python 多线程编程、GUI 界面设计以及数据可视化技术，成功打造了一个功能完备的“微型 Wireshark”。

- 主要成果如下：
- 架构解耦：成功实现了 MVC 架构，将 Scapy 的底层抓包逻辑与 PySide6 的界面展示逻辑分离，保证了系统的稳定性。
 - 深度解析：不仅实现了基础协议头的解析，还创新性地加入了 IP 地理位置定位、字段全汉化、十六进制载荷查看等实用功能。
 - 可视化分析：通过集成 Matplotlib，填补了简单抓包工具在宏观流量分析方面的

空白，提供了直观的图表报告。

通过本项目的开发，深入理解了 TCP/IP 协议栈的封装与解封装过程，掌握了网络嗅探工具的底层实现原理，并提升了 Python 全栈开发能力。

7.2 存在的不足

受限于开发时间和个人水平，系统目前仍存在以下局限性：

HTTPS 解密限制：虽然能识别 TLS 握手包，但由于缺乏中间人（MITM）证书注入机制，系统无法解密 HTTPS 的应用层加密数据，只能显示密文。

大流量处理瓶颈：Scapy 是基于 Python 的纯软件实现，在处理千兆级核心网络流量时，性能相比 C 语言编写的 Wireshark 仍有差距，可能会出现少量丢包。

过滤器功能缺失：目前尚未实现 BPF (Berkeley Packet Filter) 过滤语法，用户无法在抓包前指定只抓取特定 IP 或端口的数据。

7.3 改进方向

针对上述不足，未来的改进方向主要包括：

实现 BPF 过滤器：在界面增加过滤规则输入框（如 tcp port 80），利用 Scapy 底层过滤机制减少无用数据的捕获。

文件 I/O 支持：增加 .pcap 文件的保存与读取功能，支持离线数据包分析。

协议库扩展：增加对 FTP、SMTP、POP3 等传统应用层协议的详细命令解析支持。